

Tutorial 8

Memory management

Overview

- Computer memory
 - Definition
 - Basic execution pipeline
- Stack memory
- Variable scope
- Reference variable
- Heap memory

Computer memory: Definition

- Stores information, such as data and programs, for immediate use in the computer.
 - Main memory (RAM)
 - Directly accessible to the CPU.
 - The CPU continuously reads instructions stored there and executes them as required
 - Primary storage (Hard drive)
 - **Not** directly accessible by the CPU
 - Computer uses input/output channels to access secondary storage and **transfer** the desired data **to** primary storage

Computer memory: execution pipeline

1. Code syntax

- Follow specific formatting instructions for different programming language eg. C++
- Contained in a file eg .cpp .h for C++
 - Stored on the computer's hard drive like any other file

2. Compilation

- Convert the code syntax from human readable format to machine language
- Contained in binary file (or other formats)
 - Stored on the computer's hard drive like any other file

3. Execution

- Forwards the binary file to be interpreted by the CPU
- Contained in the RAM
 - **Stack** for predetermined data types and variables
 - **Heap** for dynamic data types

Stack memory: Description

- Stored in RAM.
 - Stack memory is temporary and automatically managed.
 - exact data needed for allocation is known before compile time.
 - It's used for local variables, function arguments, and control information.
- Uses stack data structure, FILO
 - Contiguous Blocks: Allocation occurs in contiguous blocks of memory.
 - Function Call Stack:
 - Variables are allocated on the stack when a function is called.
 - Once the function call is over, the memory is automatically deallocated.
 - Access Scope: Data stored in stack memory is accessible only within the method that contains it.
- Has a maximum size already determined when program starts.
 - Can have a **stack overflow** when too much of the stack is used:
 - infinite or too deep recursion,
 - very large allocations.

Stack memory: Variable scope

```
int myfriend(int x, string v){ // allocated on the STACK x and v

    x *= x ; // Local variable x

    return x;

} // myfriend is removed from the STACK (memory is automatically deallocated)

int main() {

    int a;           // Allocated on STACK

    int b[10];       // Allocated on STACK

    int n = 20;

    int c[n];        // Allocated on STACK

    int v = myfriend(n,"Hello!") // Allocated on the STACK, variable v and the return call

}
```

Stack memory: Reference variable

```
void myfriend(int& x, string v="123"){ // allocated on the STACK x and v
    x *= x ; // uses and updates data at memory location stored in x
} // myfriend is removed from the STACK (memory is automatically deallocated)

int main() {
    int a;           // Allocated on STACK
    int b[10];       // Allocated on STACK
    int n = 20;
    int c[n];        // Allocated on STACK
    myfriend(n,"Hello!") // Allocated on the STACK, variable v and the return call
}
```

Heap memory: Description

- Stored in RAM just like the stack.
- Allocation
 - Dynamic:
 - Created during program execution as objects are created.
 - It's used for large data structures.
 - Can have allocation failures if too big of a buffer is requested to be allocated.
 - Data created on the heap will be pointed to by pointers and allocated with **new**.
 - Objects are stored in heap space, and stack memory contains references to these objects.
 - Accessible to all functions that have reference.
- De-allocation
 - Variables on the heap must be destroyed manually and never fall out of scope.
 - The data is freed with **delete**, `delete[]`, or `free`.
 - Responsible for memory leaks.
 - Can have fragmentation when there are a lot of allocations and deallocations.

Heap memory: Example

```
int foo(){  
    char *pBuffer; //<--nothing allocated yet (excluding the pointer itself, which is  
    allocated here on the STACK).  
    bool b = true; // Allocated on the STACK.  
    if(b){  
        //Create 500 bytes on the STACK  
        char buffer[500];  
        //Create 500 bytes on the HEAP  
        pBuffer = new char[500];  
        }//<-- buffer is deallocated here, pBuffer is not  
    }  
} //<--- oops there's a memory leak, I should have called delete[] pBuffer;
```

End